

FutureSmart AI Blog

 Follow

Beginner's Guide to FastAPI & OpenAI ChatGPT Integration



Pradip Nichite

Jul 31, 2023 •  5 min read



Table of contents

Step 1: Setup

Step 2: Creating the API

Step 3: Integrating with OpenAI ChatGPT

Step 4: Running the API and Making Requests

Accessing the Swagger UI Documentation



In this guide, we'll learn how to build a Python API using FastAPI and integrate it with OpenAI's ChatGPT. By the end of this post, you'll be able to create RESTful endpoints and utilize the power of OpenAI's ChatGPT. Let's get started!

Step 1: Setup

First, we'll need to install the required packages. For this project, we're going to use FastAPI and Uvicorn for creating and running our API, and OpenAI to use the GPT-3 model for text generation.

Create a new file `requirements.txt` and add the following lines:

```
fastapi
uvicorn
openai
```

COPY

Install these packages using pip:

```
pip install -r requirements.txt
```

COPY

Step 2: Creating the API

We start by initializing our FastAPI app in the `main.py` file. Additionally, we import the Pydantic `BaseModel`, which allows us to define how our data should be modeled.

```
from fastapi import FastAPI
from pydantic import BaseModel
from utils import generate_description
```

COPY



```
app = FastAPI()
```

Next, we define our data models for the product and order. We're using Pydantic's `BaseModel` for this:

Pydantic:

Pydantic is a data validation library that uses Python type annotations. The principal advantage of Pydantic is the ease with which complex data schemas can be declared and validated. Pydantic's `BaseModel` forms the foundation for all models. It provides functionality for model initialization, serialization to JSON, model validation, etc.

Here, `Order` and `Product` are subclasses of Pydantic's `BaseModel`. By declaring our data classes in this way, Pydantic will automatically handle data validation, serialization, and documentation.

- `Order` class: This model describes the structure of an order in our application. It has two fields: `product`, which should be a string, and `units`, which should be an integer. When we create an instance of `Order`, Pydantic will ensure these types are respected, throwing an error if we attempt to assign an inappropriate value.
- `Product` class: Similarly, this model describes a product in our application. It has two fields: `name`, which should be a string, and `notes`, which should also be a string.

COPY

```
class Order(BaseModel):  
    product: str  
    units: int
```

```
class Product(BaseModel):  
    name: str  
    notes: str
```



Python Type Hints:

Python 3.5 introduced optional "type hints". You can specify the expected type of function arguments and return values. They don't affect the runtime behavior of your program but serve as documentation and are used by static type checkers, linters, and IDE features.

In the context of FastAPI, these type hints are used for:

- Data validation: For example, if we specify a function parameter to be of type `int`, and the client sends a string, FastAPI will send a helpful and descriptive error message.
- Data serialization: FastAPI can convert complex data types (like datetime objects) into formats that can be easily converted to JSON.
- API documentation: FastAPI uses these type hints to automatically generate API documentation.

So, in our `Order` and `Product` models, `str` and `int` are type hints, declaring what type of data each attribute should hold.

These concepts form the backbone of FastAPI and contribute to its fast, flexible, and developer-friendly nature.

COPY

```
class Order(BaseModel):  
    product: str  
    units: int  
  
class Product(BaseModel):  
    name: str  
    notes: str
```

With our API and data models ready, we can now create our endpoints:

1. A GET endpoint ,                                                                          

2. A GET endpoint `/hello` which takes an optional query parameter `name` and returns a personalized greeting.
3. Two POST endpoints `/orders` and `/orders_pydantic` which take product information and return a confirmation message. The difference between these two endpoints is that `/orders` takes query parameters while `/orders_pydantic` takes a JSON body.

COPY

```
@app.get("/ok")
async def ok_endpoint():
    return {"message": "ok"}

@app.get("/hello")
async def hello_endpoint(name: str = 'World'):
    return {"message": f"Hello, {name}!"}

@app.post("/orders")
async def place_order(product: str, units: int):
    return {"message": f"Order for {units} units of {product} placed successf"}

@app.post("/orders_pydantic")
async def place_order(order: Order):
    return {"message": f"Order for {order.units} units of {order.product} pla"}
```

Step 3: Integrating with OpenAI ChatGPT

We're going to use OpenAI's GPT-3 model to generate product descriptions.

In `utils.py`, we're initializing the OpenAI API key and defining a function `generate_description` which takes in product details and returns a generated description.

COPY

```
import openai

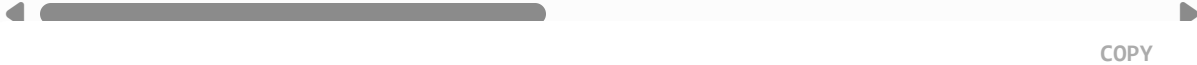
openai.api_key = "" # Add your OpenAI API key here
```

```
def generate_description(input):
    messages = [
        {"role": "user",
         "content": ""As a Product Description Generator, Generate multi par

    ]

    messages.append({"role": "user", "content": f"{input}"})
    completion = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=messages
    )
    reply = completion.choices[0].message.content
    return reply
```

Back in `main.py`, we create a POST endpoint `/product_description` which takes in product details and returns the generated description:



```
@app.post("/product_description")
async def generate_product_description(product: Product):
    description = generate_description(f"Product name: {product.name}, Notes:
    return {"product_description": description}
```

Step 4: Running the API and Making Requests

Run your API using Uvicorn:



```
uvicorn main:app --reload
```

Now you can make  examples of how to do it in Python using the requests library:

For the `/orders` endpoint, which uses query parameters:

[COPY](#)

```
import requests

url = 'http://127.0.0.1:8000/orders'
headers = {
    'accept': 'application/json',
}
params = {
    'product': 'laptop',
    'units': '1'
}

response = requests.post(url, headers=headers, params=params)

print(response.json())
```

For the `/orders_pydantic` endpoint, which uses a JSON body:

[COPY](#)

```
import requests
import json

url = 'http://127.0.0.1:8000/orders_pydantic'
headers = {
    'accept': 'application/json'
}
params = {
    'product': 'laptop',
    'units': '1'
}

response = requests.post(url, headers=headers, data=json.dumps(params))

print(response.json())
```



```
import requests
import json

url = 'http://127.0.0.1:8000/product_description'
headers = {
    'accept': 'application/json',
    'Content-Type': 'application/json'
}
data = {
    "name": "Laptop",
    "notes": "4GB RAM . 256 GB Disk"
}

response = requests.post(url, headers=headers, data=json.dumps(data))

print(response.json())
```

Accessing the Swagger UI Documentation

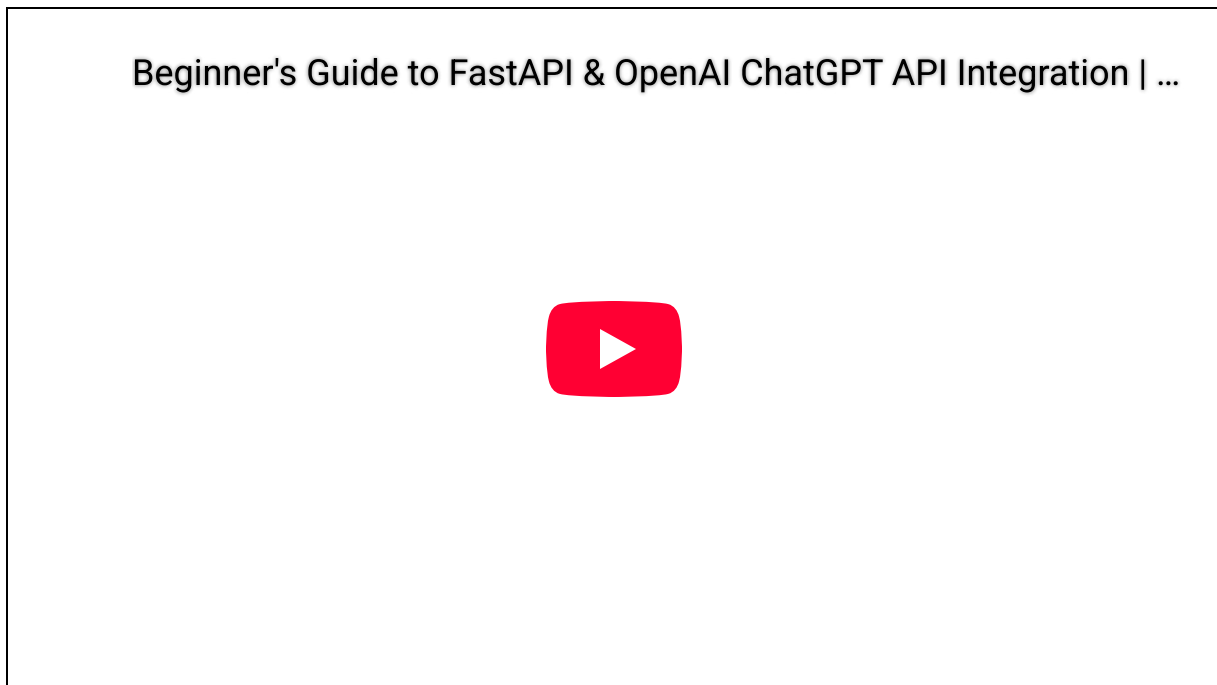
FastAPI provides out-of-the-box support for generating interactive API documentation with Swagger UI. After you've started your application using `uvicorn main:app --reload`, navigate to localhost:8000/docs in your web browser.

Here, you'll find a list of your defined endpoints (`/ok` , `/hello` , `/orders` , `/orders_pydantic` , and `/product_description`). Swagger UI allows you to experiment with your API directly from the browser: you can "Try it out", fill in the necessary fields, and then "Execute" to send a request.

This intuitive interface also presents the schemas of our Pydantic models (`Order` and `Product`), making it an excellent resource for anyone who needs to understand or interact with your API.

That's it! You now have a fully functional API with FastAPI, integrated with OpenAI's ChatGPT.  [Viewing video tutorial](#) (put the link of the video here)

Full Code : https://github.com/PradipNichite/Youtube-Tutorials/tree/main/fastapi_openai/app



FastAPI is a powerful tool for creating APIs, and when combined with OpenAI's ChatGPT, it becomes even more potent. By understanding these fundamentals, you can create more complex and robust APIs

[FastAPI](#)[openai](#)[chatgpt](#)[APIs](#)

Written by



Pradip Nichite

🚀 I'm a Top Rated Plus NLP freelancer on Upwork with over \$300K in earnings and a 100% Job Success rate. This journey began in 2022 after years of enriching  Starting my career in 2013 as a Software Developer focusing on backend and API

development, I soon pursued my interest in Data Science by earning my M.Tech in IT from IIIT Bangalore, specializing in Data Science (2016 - 2018). 📁 Upon graduation, I carved out a path in the industry as a Data Scientist at MiQ (2018 - 2020) and later ascended to the role of Lead Data Scientist at Oracle (2020 - 2022). 🌐 Inspired by my freelancing success, I founded FutureSmart AI in September 2022. We provide custom AI solutions for clients using the latest models and techniques in NLP. 🧑‍💻 In addition, I run AI Demos, a platform aimed at educating people about the latest AI tools through engaging video demonstrations. 🧰 My technical toolbox encompasses: 🛠 Languages: Python, JavaScript, SQL. 📄 ML Libraries: PyTorch, Transformers, LangChain. 🔍 Specialties: Semantic Search, Sentence Transformers, Vector Databases. 💻 Web Frameworks: FastAPI, Streamlit, Anvil. ☁ Other: AWS, AWS RDS, MySQL. 🚀 In the fast-evolving landscape of AI, FutureSmart AI and I stand at the forefront, delivering cutting-edge, custom NLP solutions to clients across various industries.

[Follow](#)

Published on



FutureSmart AI Blog

FutureSmart AI provides custom Natural Language Processing (NLP) solutions.

[Follow](#)

MORE ARTICLES



Pradip Nichite

Building AI Knowledge Graph Using Graphiti & [REDACTED]



Graphiti is a compact yet powerful Python library that converts raw text or JSON into an AI Knowledgeg...



Pradip Nichite



LangGraph +



Add Long-Term Memory to AI Agents

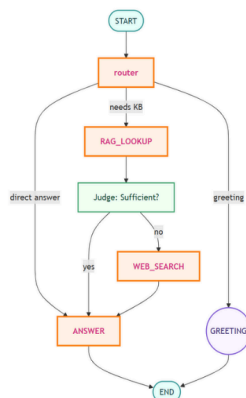
AI Agents Memory: Mem0 + LangGraph Agent Integration

In this blog we'll walk through practical steps to add long-term memory to your AI agents using Mem0...



Pradip Nichite

LangGraph RAG Agent



LangGraph to
Advanced Multi-Agent AI Chatbot

Retrieval-Augmented Generation (RAG) is becoming the go-to pattern for building AI systems that can ...

©2025 FutureSmart AI Blog

[Archive](#) • [Privacy policy](#) • [Terms](#)



Powered by Hashnode - Build your developer hub.

[Start your blog](#)

[Create docs](#)

